

Aufgabe 1: Java-Programmierung**(16 Punkte)**

Gegeben ist nachfolgendes, kompilierbare Programm:

```
class A {
    int i;
    B b;
    A(int i, B b) {
        this.i = i;
        this.b = b;
    }
}

class B {
    int i;
    B(int i) {
        this.i = i;
    }
}

class M {
    M() {
        System.out.println("M.M()");
    }
}

class X {
    X() {
        System.out.println("X.X()");
    }
    void meth() {
        System.out.println("X.meth()");
    }
}

class Y extends X {
    Y() {
        System.out.println("Y.Y()");
    }
    void meth() {
        System.out.println("Y.meth()");
    }
}

M m = new M();
```

```

public class JavaProgramming {
    public static void main(String[] args) {
        A a1 = new A(1, new B(2));
        A a2 = a1;
        a2.i = 3;
        System.out.println("a1.i = " + a1.i);
        A a3 = new A(4, new B(5));
        B b = new B(6);
        method(a3, b);
        System.out.println("a3.i = " + a3.i);
        System.out.println("a3.b.i = " + a3.b.i);
        System.out.println("b.i = " + b.i);
        X x = new Y();
        x.meth();
    }

    static void method(A a, B b) {
        a.i = 9;
        a.b = new B(10);
        b.i = 11;
        b = new B(12);
    }
}

```

Was ist die Ausgabe des Programmes auf der Konsole wenn main() zur Ausführung gebracht wird? :

a1.i = 3 2

a3.i = 9 2

a3.b.i = ~~5~~

b.i = 11 2

X.X() 4

M.HH) Y.Y()

Y.meth() 2

Aufgabe 2: Arithmetische Reihe**(13 Punkte)**

Bestimmen Sie die Summenformel ($s_n = \sum_{i=1}^n a_i$) der folgenden Folge in *rekursiver*, *iterativer* und *expliziter* Form.

Die Polynome jeweils in der Normalform: $a_n x^n + a_{n-1} x^{n-1} + \dots + a_2 x^2 + a_1 x + a_0$

Die Folge: 5, 14, 23, 32, ...
 $\begin{matrix} & & \nearrow 9 \\ +9 & \nearrow & 9 \end{matrix}$

$$n=1 \quad 9 \cdot n + x = 5$$

$$9 + x = 5$$

$$9 - 3 = 5$$

Rekursiv: $s_n = s_{n-1} + 9n - 3$ ⁴ $s_1 = 5$

Iterativ: $\sum_{i=1}^n 9i - 3$ ⁴

Explizit: $\frac{n(9n - 3 + 5)}{2} = \frac{n(9n + 2)}{2} = \frac{2n + 9n^2}{2}$

$$= \frac{9n^2}{2} + \frac{n}{2}$$

Folgelektor

Aufgabe 3: Vollständige Induktion**(16 Punkte)**

Es soll mit einer 'vollständigen Induktion' folgende Behauptung bewiesen werden:

$$14 + \sum_{i=1}^n (4i+9) = 2n^2 + 11n + 14$$

① $n=1$ $14 + (4 \cdot 1 + 9) = 2 \cdot 1^2 + 11 \cdot 1 + 14$
 $27 = 27$ Verankerung ok

② $14 + \sum_{i=1}^n (4i+9) = 2n^2 + 11n + 14$

③ Behauptung: $14 + \sum_{i=1}^{n+1} (4i+9) = 2 \cdot (n+1)^2 + 11(n+1) + 14$ ✓

④ Beweis

$$2(n+1)^2 + 11(n+1) + 14 = 14 + 4(n+1) + 9 + 2n^2 + 11n + 4$$

$$2(n^2 + 2n + 1) + 11n + 11 + 14 = 14 + 4n + 4 + 9 + 2n^2 + 11n + 4$$

$$14 + 4 + 9 + 4 = 14 + 15 + 4$$

$$2n^2 + 15n + 27 = 2n^2 + 15n + 27$$

Aufgabe 4: Laufzeit-Analyse**(16 Punkte)**

Analysieren Sie die Laufzeit des folgenden Programmfragments:

```

for (int i = 0; i <= 2 * n + 2; i = i + 2) {
    for (int j = 0; j < i + 3; j++) {
        meth();
    }
}

```

$n=4$ bis 2 mit 8
 äusser $i = 0, 2, 4, 6, 8$
 innere $3 \times, 5 \times, 7 \times, 9 \times, 11 \times$
 $0, 1, 2 < 3$
 $0, 1, 2, 3, 4 < 2+3=5$

Wobei die Laufzeit von `meth()` konstant ist ($O(1)$).Geben Sie die Laufzeit in Abhängigkeit von n in *expliziter* Form an.Die *explizite* Form in der Polynom-Normalform: $a_n x^n + a_{n-1} x^{n-1} + \dots + a_2 x^2 + a_1 x + a_0$

$$4+3=7 \quad 0, 1, 2, 3, 4, 5, 6$$

$$m = n+1 \quad n+2$$

$$d = 2 \quad 4$$

$$a_1 = 3 \quad 4$$

$$\sum a_i = a_1 m + \frac{m(m-1)}{2} \cdot d$$

$$3 \cdot (n+1) + \frac{(n+1)(n+1)}{2} \cdot 2$$

$$= 3n+3 + \frac{n^2+n}{2} \cdot 2$$

$$= 3n+3 + n^2+n = \underline{\underline{n^2 + 4n + 3}}$$

Aufgabe 6: Rekursion**(10 Punkte)**

Gegeben ist folgendes, fehlerfreie Code-Segment:

```

static int recursion(int i1, int i2) {
    if (i1 <= 4) {
        System.out.println(i1 + "/" + i2);
        return 5;
    }
    switch(i1) {
        case 8:
            return 1 + recursion(i1 - 1, i2 + 1);
        case 5:
            return 2 + recursion(i1 - 4, i2 + 2);
        default:
            return 3 + recursion(i1 - 2, i2 + 3);
    }
}

public static void main(String[] args) {
    System.out.println(recursion(12, 1));
}

```

kein Break

12, 1

Was ist die Ausgabe auf der Konsole wenn `main()` zur Ausführung gebracht wird? $rec(12, 1)$ $3 + rec(10, 4)$ $3 + rec(8, 7)$ $1 + rec(7, 8)$ $2 + rec(5, 11)$ $2 + rec(1, 13)$ $return 5$

1 / 13 ✓	5
15	

Gegeben ist die Schnittstelle für eine Methode:
`void sort(List<Integer> list)`

Eine implementierende Methode dieser Schnittstelle hat folgende Laufzeitverhalten bezüglich dreier Varianten der Input-Daten (Vorsortierung) und in Abhängigkeit der Länge n der übergebenen Liste:

Variante	Laufzeitverhalten (explizit)
1	$(n+3) \cdot (n-1)$
2	$1000 \cdot n$
3	$n \cdot \log(n)$

$\rightarrow O(n^2)$

Eine Laufzeitmessung mit Variante 1 und 1'000 Elementen ergibt: 100 Millisekunden

Welche Laufzeiten sind zu erwarten bei folgenden Konstellationen:

Variante	Länge n der Liste	zu erwartende Laufzeit in Millisekunden als eine Zahl (nicht als Formel)?
1	4'000	1600 ms
2	1'000	100 ms
3	1'000	1 ms
3	8'000	~ 1.7 ms

$\cdot 4 \quad 4^2 \cdot 100 \text{ ms} = 16 \cdot 100 \text{ ms}$

2) $\frac{100 \text{ ms}}{(n+3) \cdot (n-1)} \cdot 1000 \cdot n \approx \frac{100 \text{ ms}}{1000 \cdot 1000} \cdot 1000 \cdot 1000 \approx 100 \text{ ms}$

3) $\frac{100 \text{ ms}}{(n+3) \cdot (n-1)} \cdot n \cdot \log(n) \approx \frac{100 \text{ ms}}{1000 \cdot 1000} \cdot 1000 \cdot \log_2(1000)$
 $2^{10} \approx 1024 \rightarrow 10$
 $= \frac{100 \text{ ms} \cdot 10}{1000} = 1 \text{ ms}$

4) $\frac{100 \text{ ms}}{8000 \cdot 8000} \cdot 8000 \cdot \log_2(8000) = \frac{13000}{8000} \approx 1.7 \text{ ms}$
 $2^{13} \approx 8192 \rightarrow 13$

Aufgabe 5: Laufzeiten

(16 Punkte)

Gegeben ist die Schnittstelle für eine Methode:
`void sort(List<Integer> list)`

Eine implementierende Methode dieser Schnittstelle hat folgende Laufzeitverhalten bezüglich dreier Varianten der Input-Daten (Vorsortierung) und in Abhängigkeit der Länge n der übergebenen Liste:

Variante	Laufzeitverhalten (explizit)
1	$(n+3) \cdot (n-1)$
2	$1000 \cdot n$
3	$n \cdot \log(n)$

$\rightarrow O(n^2)$

Eine Laufzeitmessung mit Variante 1 und 1'000 Elementen ergibt: 100 Millisekunden

Welche Laufzeiten sind zu erwarten bei folgenden Konstellationen:

Variante	Länge n der Liste	zu erwartende Laufzeit in Millisekunden als eine Zahl (nicht als Formel)?
1	4'000	1600 ms
2	1'000	100 ms
3	1'000	1 ms
3	8'000	≈ 1.7 ms

$\cdot 4 \quad 4^2 \cdot 100 \text{ ms} = 16 \cdot 100 \text{ ms}$

2) $\frac{100 \text{ ms}}{(n+3) \cdot (n-1)} \cdot 1000 \cdot n \approx \frac{100 \text{ ms}}{1000 \cdot 1000} \cdot 1000 \cdot 1000 \approx 100 \text{ ms}$

3) $\frac{100 \text{ ms}}{(n+3) \cdot (n-1)} \cdot n \cdot \log(n) \approx \frac{100 \text{ ms}}{1000 \cdot 1000} \cdot 1000 \cdot \log_2(1000)$
 $2^{10} \approx 1024 \rightarrow 10$
 $= \frac{100 \text{ ms} \cdot 10}{1000} = 1 \text{ ms}$

4) $\frac{100 \text{ ms}}{8000 \cdot 8000} \cdot 8000 \cdot \log_2(8000) = \frac{13000}{8000} = 1.625 \approx 1.7 \text{ ms}$
 $2^{13} \approx 8192$

12

Aufgabe 7: Iteratoren

(18 Punkte)

Gegeben sind die nachfolgenden drei kompilierbaren Methoden.
Notieren Sie die jeweilige Ausgabe auf der Konsole wenn die Methoden zur Ausführung gebracht werden.

Hinweis: Exceptions werden nirgends (mit catch) abgefangen.

```
void method1() {
    java.util.List<Integer> list = new java.util.LinkedList<>();
    for (int i = 1; i < 7; i++) {
        list.add(i);
    }
    java.util.Iterator<Integer> it = list.iterator();
    for (int i = 0; i < 3; i++) {
        it.next();
    }
    it.remove();
    System.out.println(it.hasNext());
    it.remove();
    while(it.hasNext()) {
        System.out.print(it.next() + " ");
    }
}
```

Handwritten notes: 1, 2, 3, 4, 5, 6 (under list.add(i));)

Konsolen-Ausgabe:

~~6~~

~~it.next() + null~~

Index 0

```
void method2() {
    java.util.List<Integer> list = new java.util.LinkedList<>();
    for (int i = 0; i < 4; i++) {
        list.add(i);
    }
    java.util.Iterator<Integer> it = list.iterator();
    it.next();
    it.next();
    list.add(4);
    boolean hasNext = it.hasNext();
    while(hasNext) {
        System.out.println(hasNext + " ");
        System.out.println(it.next() + " ");
        hasNext = it.hasNext();
    }
}
```

Handwritten notes: 0, 1, 2, 3, 4 (under list.add(i));)

Error

Konsolen-Ausgabe:

~~No Object bei while (hasNext)~~

~~Sollte it.hasNext() sein~~

```
void method3() {  
    java.util.List<Integer> list = new java.util.ArrayList<>();  
    for (int i = 1; i <= 5; i++) {  
        list.add(i);  
    }  
    java.util.ListIterator<Integer> it = list.listIterator();  
    int i = 1;  
    while (it.hasNext()) {  
        it.next();  
        if (i == list.size() - 3) {  
            it.previous();  
            it.add(-1);  
            break;  
        }  
        i++;  
    }  
    it = list.listIterator();  
    while (it.hasNext()) {  
        System.out.print(it.next() + " ");  
    }  
}
```

Handwritten notes in the code:
- Above `list.add(i);`: 1, 2, 3, 4, 5
- Above `if (i == list.size() - 3)`: 2, 3
- Next to `it.previous();`: $1 \rightarrow 4 - 3 = 1$
- Next to `it.add(-1);`: 1, -1, 2, 3, 4, 5
- An arrow points from the `break;` statement to the `it = list.listIterator();` line.

Konsolen-Ausgabe:

Handwritten output: 1, -1, 2, 3, 4, 5 ✓

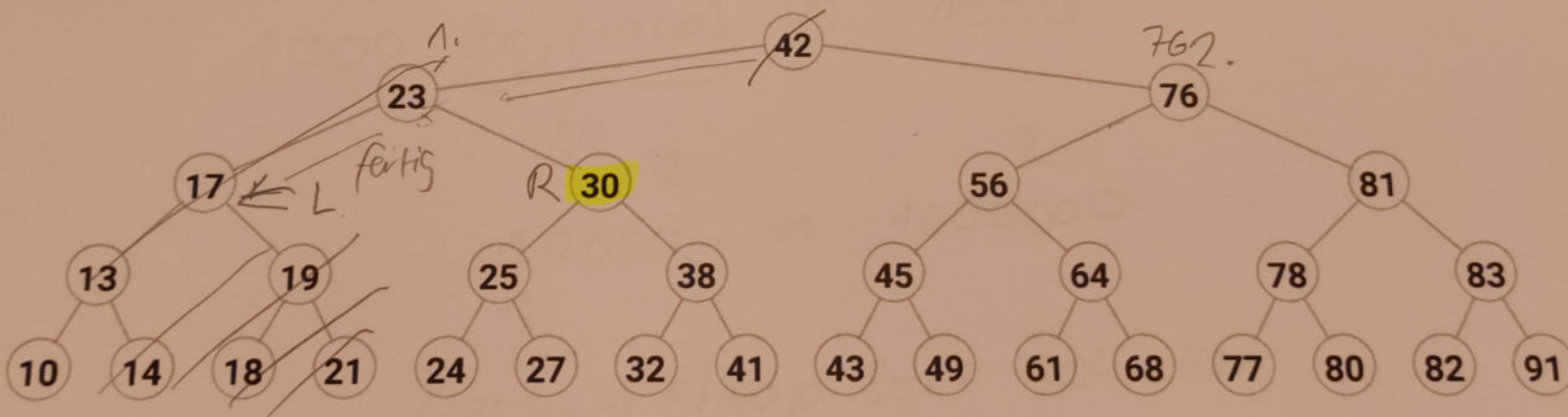
Aufgabe 8: Baum-Traversierung**(8 Punkte)**

Gegeben ist folgender Algorithmus für die *Breadth-First Traversierung* eines Baumes:

```

Algorithm breadthFirst()
  Initialize queue Q containing root
  while Q not empty do
    v = Q.dequeue()
    visit (v)
    for each child w in children(v) do
      Q.enqueue(w)
  
```

Im Weiteren folgender Baum auf welchem der Algorithmus angewendet wird:



Hinweis: Der *for-each-Loop* liefert jeweils zuerst das linke und dann das rechte Kind.

Was ist der Inhalt der **Queue Q**, nachdem der while-Loop, bei welchem der Knoten **30** besucht/bearbeitet wurde, beendet ist?

25

(42, 23, 76, ~~17~~, 30), 56, 81, 13, 19, 21

→ 56, 81, ¹³25, 38, ~~38~~

Aufgabe 8: Baum-Traversierung

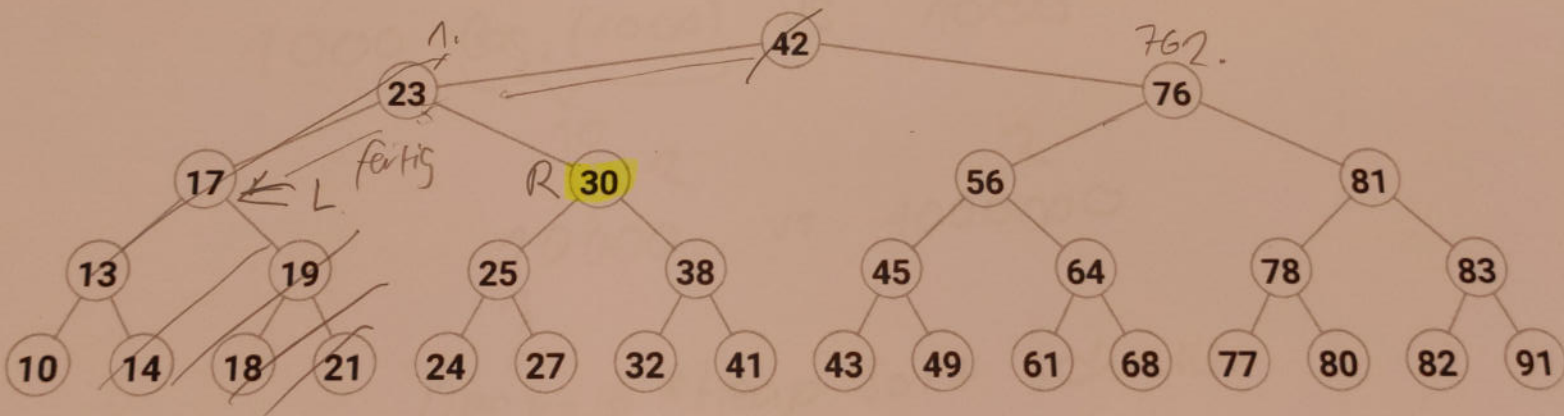
(8 Punkte)

Gegeben ist folgender Algorithmus für die *Breadth-First Traversierung* eines Baumes:

```

Algorithm breadthFirst()
  Initialize queue Q containing root
  while Q not empty do
    v = Q.dequeue()
    visit (v)
    for each child w in children(v) do
      Q.enqueue(w)
  
```

Im Weiteren folgender Baum auf welchem der Algorithmus angewendet wird:



Hinweis: Der *for-each-Loop* liefert jeweils zuerst das linke und dann das rechte Kind.

Was ist der Inhalt der **Queue Q**, nachdem der *while-Loop*, bei welchem der Knoten **30** besucht/bearbeitet wurde, beendet ist?

25 (42, 23, 76, ~~23~~ 17, 30), 56, 81, 13, 19, 25

→ 56, 81, ¹³25, 38, ~~38~~

Aufgabe 9: Sortierung

Deque Selection Sort
 $O(n^2)$ (9 Punkte)

Eine Liste mit 1'000 Elementen wird mit Selection-Sort sortiert.
 Wie viel mal schneller oder langsamer ist die Sortierung, wenn anstelle des Selection-Sort ein Heap-Sort angewendet wird?

Begründen Sie ihre Antwort.

Heap-Sort: $O(n \log n)$ 2

Deque Selection Sort $O(n^2)$ 2

Vergleich

$$1000 \cdot \underbrace{\log_2(1000)}_{10} \text{ VS. } 1000^2$$

$$10'000 \text{ VS. } 1'000'000$$

↑
 Deshalb ist Heap Sort schneller

8

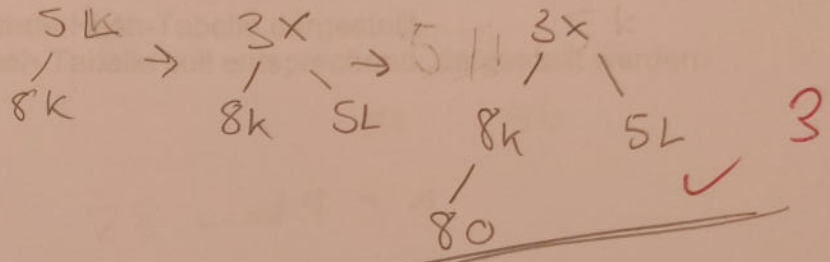
Aufgabe 10: Heap-basierte Adaptable-Priority-Queue**(12 Punkte)**

Gegeben sind nachfolgend drei Code-Sequenzen welche jeweils mit der *heap-basierten Adaptable-Priority-Queue* **HeapAPQ** arbeiten.

Zeichnen Sie bei den drei Code-Sequenzen jeweils den Endzustand des Heaps auf (die Zwischenschritte sind nicht nötig).

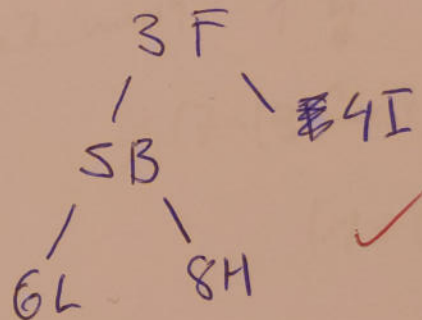
1. Code-Sequenz:

```
HeapAPQ<Integer, String> apq = new HeapAPQ<>();
apq.insert(8, "K");
apq.insert(5, "L");
apq.insert(3, "X");
apq.insert(8, "O");
```



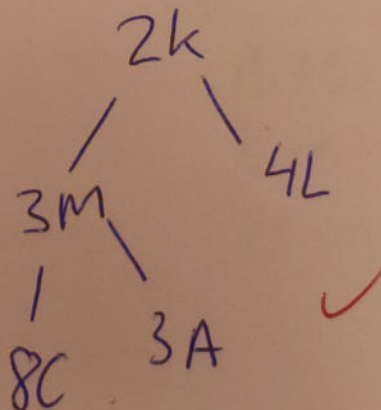
2. Code-Sequenz:

```
HeapAPQ<Integer, String> apq = new HeapAPQ<>();
apq.insert(1, "G");
apq.insert(3, "F");
apq.insert(4, "I");
apq.insert(6, "L");
apq.insert(5, "B");
apq.insert(8, "H");
apq.removeMin();
```



3. Code-Sequenz:

```
HeapAPQ<Integer, String> apq = new HeapAPQ<>();
Entry<Integer, String> e1 = apq.insert(8, "C");
Entry<Integer, String> e2 = apq.insert(2, "K");
Entry<Integer, String> e3 = apq.insert(1, "L");
Entry<Integer, String> e4 = apq.insert(3, "M");
Entry<Integer, String> e5 = apq.insert(3, "A");
apq.replaceKey(e3, 4);
```



72

Aufgabe 11: Hash-Tabelle**(10 Punkte)**

Gegeben ist eine Hash-Tabelle mit folgenden Eigenschaften:

- Offene Adressierung
- Doppeltes Hashing
- 1. Hash-Funktion: $h(k) = k \bmod 9$
- 2. Hash-Funktion: $d(k) = 7 - (k \bmod 7)$ $N=9$ $(h(k) + c \cdot d(k)) \bmod 9$

Nun werden der Reihe nach folgende Zahlen hinzugefügt:
28, 14, 37, 10, 23 (zuerst die 28)

Nachfolgend ist die entsprechende Hash-Tabelle dargestellt.

Der resultierende Inhalt der Hash-Tabelle soll entsprechend dargestellt werden:

h(i)	Double Hashing
0	10
1	28
2	23
3	
4	10
5	14
6	37
7	37
8	

$$28 \bmod 9 = 1$$

$$14 \bmod 9 = 5$$

$$37 \bmod 9 = 1 \downarrow$$

$$(7 - (37 \bmod 7)) = 5$$

$$(1 + 5) \bmod 9 = 6 \text{ ok}$$

$$10 \bmod 9 = 1 \downarrow$$

$$7 - (10 \bmod 7) = 4 \parallel 1 + 4 = 5$$

$$5 \bmod 9 = 5$$

$$(1 + 8) \bmod 9 = 0$$

$$23 \bmod 9 = 5 \downarrow$$

$$7 - (23 \bmod 7) = 5$$

21

$$(5 + 5) \bmod 9 = 1 \downarrow$$

$$(5 + 10) \bmod 9 = 6 \downarrow$$

$$(5 + 15) \bmod 9 = 2$$

18

10

Aufgabe 12: Klassen-Design und Implementation

(28 Punkte)

Gegeben sind die folgenden, korrekt funktionierenden Klassen, welche eine unsortierte Menge von Werten (*value's*) in einem Array von Entries bewirtschaften, welche mit der `add()` -Methode hinzugefügt wurden:

```
class MySimpleEntryList {
    static final int SIZE = 100000;

    protected Entry[] entries = new Entry[SIZE];
    protected int ci; // the current index in the for-loops

    public Entry add(int value) {
        for (ci = 0; ci < entries.length; ci++) {
            if (entries[ci] == null) {
                Entry entry = newEntry(value);
                entries[ci] = entry;
                return entry;
            }
        }
        return null;
    }

    public Integer remove(Entry entry) {
        for (ci = 0; ci < entries.length; ci++) {
            if (entries[ci] == entry) {
                entries[ci] = null;
                return entry.value;
            }
        }
        return null;
    }

    protected Entry newEntry(int value) {
        return new Entry(value);
    }

    public static void main(String[] args) {
        MySimpleEntryList myEntries = new MySimpleEntryList();
        Entry entry = myEntries.add(11);
        Integer value = myEntries.remove(entry);
        System.out.println(value); // Session-Log in Console: 11
        value = myEntries.remove(entry);
        System.out.println(value); // Session-Log in Console: null
    }
}

class Entry {
    int value;

    Entry(int value) {
        this.value = value;
    }
}
```

Handwritten notes:

- Next to `add()`: $e1=11$, $e2=2$, $e3=3$
- Next to `remove()`: $O(n) \rightarrow O(1)$

Die `remove()`-Methode hat somit eine Laufzeit von $O(n)$.

Nun soll die Klasse `MyFastRemoveEntryList` erstellt werden, bei welcher die `remove()`-Methode eine Laufzeit von nur noch $O(1)$ hat.

Bedingungen:

- Die Klasse `MyFastRemoveEntryList` ist eine Ableitung von `MySimpleEntryList`
(`class MyFastRemoveEntryList extends MySimpleEntryList`)
- Es soll soviel wie möglich von `MySimpleEntryList` wiederverwendet werden (d.h. es sollen keine zusätzlichen neuen Datenstrukturen, wie z.B. Hash-Table, verwendet werden oder kein unnötig duplizierter Source-Code erstellt werden).
- Die bestehenden Klassen dürfen **NICHT** verändert werden, **ausser** in `main()` die Zeile mit der Instanzierung des Listen-Objektes:
`MySimpleEntryList myEntries = new MyFastRemoveEntryList();`
- Die `main()`-Methode verhält sich funktional gleich wie vorher.
- Es können neben der Klasse `MyFastRemoveEntryList` auch noch weitere neue Klassen erstellt werden.

Tipp: Location-Awareness

Erstellen Sie nun die Erweiterungen:

```
class MyFastRemoveEntryList extends MySimpleEntryList {
```

```
    Iterator it = entries.iterator();
```

```
    @Override
```

```
    public Integer remove(Entry entry) {
```

```
        while (it.hasNext()) {
```

```
            Entry temp = it.next();
```

```
            if (temp == entry) {
```

```
                int temp = it.value;
```

```
                entries[it] = null;
```

```
                return
```

```
                return temp;
```

```
            }
```

```
        }
```

```
        return null;
```

```
    }
```

5